BeSafe Hackathon

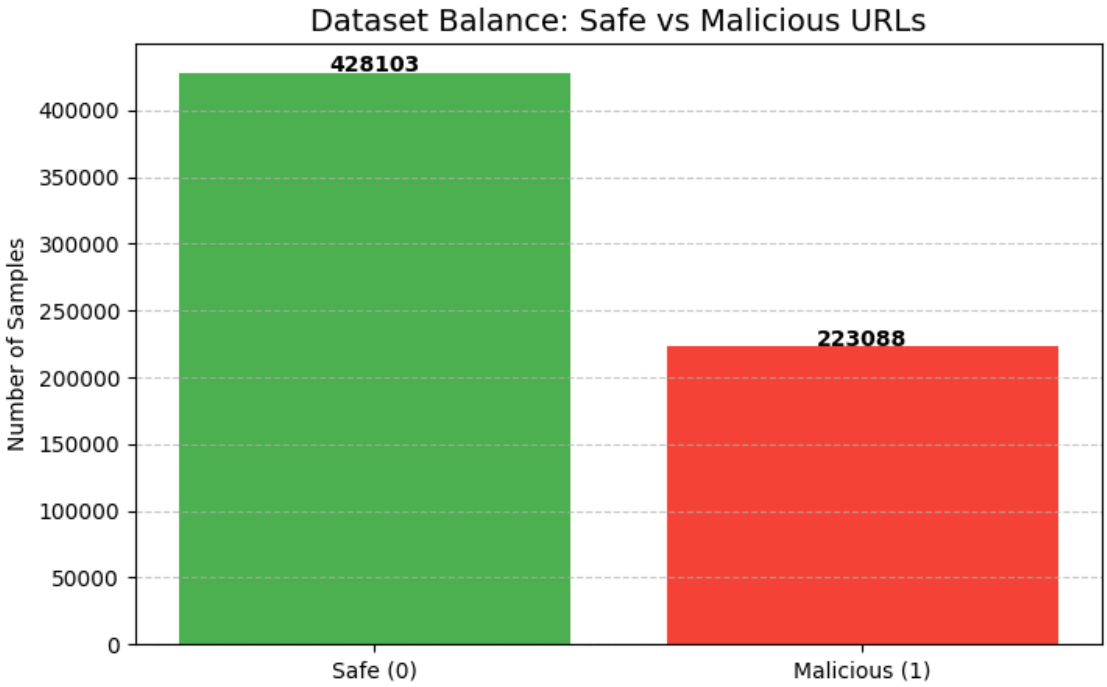
January 11, 2026

[1]:
```
import pandas as pd
import numpy as np
import xgboost as xgb
import matplotlib.pyplot as plt
import kagglehub
import os
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.metrics import accuracy_score

# 1. Load Data
# ------------------------------------------------------------
if 'df' not in locals():
    print("Downloading dataset...")
    path = kagglehub.dataset_download("sid321axn/malicious-urls-dataset")
    csv_file = [f for f in os.listdir(path) if f.endswith('.csv')][0]
    full_path = os.path.join(path, csv_file)
    print(f"Loading CSV from: {full_path}")
    df = pd.read_csv(full_path)

# 2. Encode Labels
# ------------------------------------------------------------
# 0 = Safe (benign), 1 = Malicious (defacement, phishing, malware)
df['label'] = df['type'].apply(lambda x: 0 if x == 'benign' else 1)

# 3. Visualize Data Balance (Critical Step)
# ------------------------------------------------------------
label_counts = df['label'].value_counts()
print("Data Distribution:\n", label_counts)

plt.figure(figsize=(8, 5))
# Plotting bar chart of Benign vs Malicious
colors = ['#4CAF50', '#F44336'] # Green for Safe, Red for Bad
plt.bar(['Safe (0)', 'Malicious (1)'], label_counts.values, color=colors)
plt.title('Dataset Balance: Safe vs Malicious URLs', fontsize=14)
plt.ylabel('Number of Samples')
plt.grid(axis='y', linestyle='--', alpha=0.7)
```

```
# Add text on top of bars
for i, v in enumerate(label_counts.values):
    plt.text(i, v + 500, str(v), ha='center', fontweight='bold')

plt.show()

# Planner Note: If the red bar is much lower than the green bar,
# we have an "Imbalanced Dataset". XGBoost has a fix for this
# called 'scale_pos_weight', which we will use later.
```

```
C:\Users\lina3\AppData\Local\Programs\Python\Python313\Lib\site-
packages\tqdm\auto.py:21: TqdmWarning: IProgress not found. Please update
jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

```
Downloading dataset…
Warning: Looks like you're using an outdated `kagglehub` version (installed:
0.3.13), please consider upgrading to the latest version (0.4.0).
Loading CSV from: C:\Users\lina3\.cache\kagglehub\datasets\sid321axn\malicious-
urls-dataset\versions\1\malicious_phish.csv
Data Distribution:
 label
0    428103
1    223088
Name: count, dtype: int64
```



[2]:
```
print("--- Data Overview ---")
print(f"Total Rows: {len(df)}")
print("\n--- Labels Distribution (Original) ---")
print(df['type'].value_counts())

print("\n--- Labels Distribution (Percentage) ---")
print(df['type'].value_counts(normalize=True) * 100)

print("\n--- Sample Data (First 5 rows) ---")
display(df.head()) # display       Jupyter-
```

```
--- Data Overview ---
Total Rows: 651191

--- Labels Distribution (Original) ---
type
benign        428103
defacement     96457
phishing       94111
malware        32520
Name: count, dtype: int64

--- Labels Distribution (Percentage) ---
type
benign        65.741541
defacement    14.812398
phishing      14.452135
malware        4.993927
Name: proportion, dtype: float64

--- Sample Data (First 5 rows) ---
```

	url	type	label
0	br-icloud.com.br	phishing	1
1	mp3raid.com/music/krizz_kaliko.html	benign	0
2	bopsecrets.org/rexroth/cr/1.htm	benign	0
3	http://www.garage-pirenne.be/index.php?option=…	defacement	1
4	http://adventure-nicaragua.net/index.php?optio…	defacement	1

[3]:
```
import re
from urllib.parse import urlparse

def advanced_feature_extraction(url):
    url = str(url)
    features = []
```

```
    # --- Structural Features (The Shape of the URL) ---

    # 1. Total Length
    features.append(len(url))

    # 2. Count Dots (Phishing often uses many subdomains: a.b.c.bank.com)
    features.append(url.count('.'))

    # 3. Count Hyphens (Legit sites rarely use: secure-login-account-update)
    features.append(url.count('-'))

    # 4. Count Special Obfuscation Chars (@, %, // in middle)
    features.append(url.count('@'))
    features.append(url.count('%'))

    # 5. Directory Depth (How many slashes after the domain?)
    # http://com/a/b/c/d/e -> depth 5
    features.append(url.count('/'))

    # --- Statistical Features (Math on the string) ---

    # 6. Digit to Letter Ratio (Malware links often have random numbers)
    digits = sum(c.isdigit() for c in url)
    letters = sum(c.isalpha() for c in url)
    features.append(digits / (letters + 1)) # +1 to avoid division by zero

    # 7. Has IP Address? (e.g. http://192.168.1.1/login)
    # Using a simple Regex to find IP patterns
    has_ip = 1 if re.search(r'\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}', url) else 0
    features.append(has_ip)

    # --- Keyword Features (The "Intent") ---

    # 8. Suspicious Keywords (Financial/Urgency)
    suspicious_words = ['login', 'signin', 'bank', 'secure', 'account',␣
↪'update',
                        'verify', 'confirm', 'wallet', 'crypto', 'bonus',␣
↪'free']
    features.append(sum(1 for word in suspicious_words if word in url.lower()))

    # 9. Brand Spoofing (Commonly targeted brands)
    brands = ['paypal', 'apple', 'google', 'microsoft', 'facebook', 'netflix',␣
↪'amazon']
    features.append(sum(1 for brand in brands if brand in url.lower()))

    # 10. File Extensions (Is it trying to download something?)
    dangerous_ext = ['.exe', '.zip', '.tar', '.js', '.vbs', '.apk']
```

```
    features.append(1 if any(ext in url.lower() for ext in dangerous_ext) else␣
 ↪0)

    # 11. Is HTTPS? (0 if http, 1 if https - though many phishers use https now)
    features.append(1 if 'https' in url.lower() else 0)

    # 12. "www" presence (Random generated domains often skip 'www')
    features.append(1 if 'www.' in url.lower() else 0)

    return features

# Apply to DataFrame
print("Extracting features... (This might take a minute)")
X_temp = df['url'].apply(lambda x: advanced_feature_extraction(x))

# Convert list of lists to DataFrame
feature_names = [
    'len', 'dots', 'hyphens', 'at_symbol', 'percent', 'slashes',
    'digit_ratio', 'has_ip', 'sus_keywords', 'brands', 'is_file', 'is_https',␣
 ↪'has_www'
]
X = pd.DataFrame(X_temp.tolist(), columns=feature_names)
y = df['label']

print("Feature Matrix Shape:", X.shape)
print("First 5 rows of features:")
display(X.head()) # Or print(X.head()) if not in Jupyter
```

```
Extracting features… (This might take a minute)
Feature Matrix Shape: (651191, 13)
First 5 rows of features:
```

	len	dots	hyphens	at_symbol	percent	slashes	digit_ratio	has_ip	\
0	16	2	1	0	0	0	0.000000	0	
1	35	2	0	0	0	2	0.033333	0	
2	31	2	0	0	0	3	0.038462	0	
3	88	3	1	0	0	3	0.109375	0	
4	235	2	1	0	0	3	0.110000	0	

	sus_keywords	brands	is_file	is_https	has_www
0	0	0	0	0	0
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	1
4	0	0	0	0	0

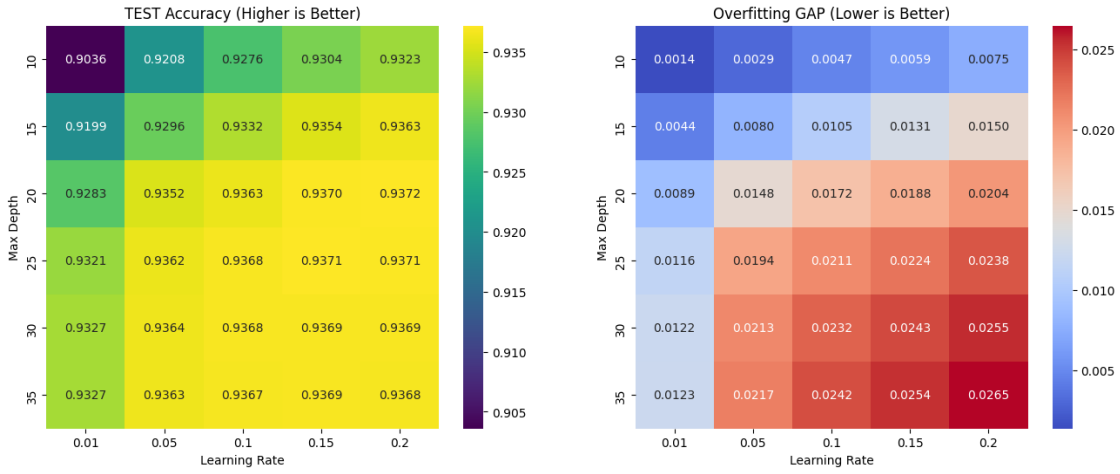
[14]:
```
import seaborn as sns
import pandas as pd
```

```
import matplotlib.pyplot as plt
import xgboost as xgb
from sklearn.model_selection import cross_validate # <--- The new tool

# 1. Search Ranges
param_grid = {
    'max_depth': [10, 15, 20, 25, 30, 35],
    'learning_rate': [0.01, 0.05, 0.1, 0.15, 0.2]
}

results = []

print("Starting Deep Dive Grid Search (Train vs Test)...")
print("-----------------------------------------------------------------")
print(f"{'Depth':<6} | {'Rate':<6} | {'Train Acc':<10} | {'Test Acc':<10} |␣
 ↪{'Gap (Overfit)':<12}")
print("-----------------------------------------------------------------")

# 2. Double Loop
for depth in param_grid['max_depth']:
    for rate in param_grid['learning_rate']:

        model = xgb.XGBClassifier(
            n_estimators=100,
            max_depth=depth,
            learning_rate=rate,
            eval_metric='logloss'
        )

        # Use cross_validate to get BOTH scores
        cv_results = cross_validate(
            model,
            X_train,
            y_train,
            cv=3,
            scoring='accuracy',
            return_train_score=True # <--- This is the magic key
        )

        # Calculate averages
        mean_train = cv_results['train_score'].mean()
        mean_test = cv_results['test_score'].mean()
        gap = mean_train - mean_test

        results.append([depth, rate, mean_train, mean_test, gap])

        # Print row-by-row
```

```
        print(f"{depth:<6} | {rate:<6} | {mean_train*100:.2f}%      |␣
 ↪{mean_test*100:.2f}%      | {gap*100:.2f}%")

print("-----------------------------------------------------------------")

# 3. Visualize Results
df_results = pd.DataFrame(results, columns=['Max Depth', 'Learning Rate',␣
 ↪'Train Accuracy', 'Test Accuracy', 'Gap'])

# Create two heatmaps side-by-side
fig, ax = plt.subplots(1, 2, figsize=(16, 6))

# Heatmap 1: Test Accuracy (What we want to maximize)
pivot_test = df_results.pivot(index='Max Depth', columns='Learning Rate',␣
 ↪values='Test Accuracy')
sns.heatmap(pivot_test, annot=True, fmt=".4f", cmap='viridis', ax=ax[0])
ax[0].set_title('TEST Accuracy (Higher is Better)')

# Heatmap 2: Overfitting Gap (What we want to minimize)
pivot_gap = df_results.pivot(index='Max Depth', columns='Learning Rate',␣
 ↪values='Gap')
sns.heatmap(pivot_gap, annot=True, fmt=".4f", cmap='coolwarm', ax=ax[1])
ax[1].set_title('Overfitting GAP (Lower is Better)')

plt.show()

# 4. Auto-select the winner (Based on best TEST accuracy)
best_row = df_results.loc[df_results['Test Accuracy'].idxmax()]
print("\n WINNER PARAMETERS:")
print(f"Best Depth: {int(best_row['Max Depth'])}")
print(f"Best Learning Rate: {best_row['Learning Rate']}")
print(f"Test Accuracy: {best_row['Test Accuracy']*100:.2f}%")
print(f"Train Accuracy: {best_row['Train Accuracy']*100:.2f}% (Gap:␣
 ↪{best_row['Gap']*100:.2f}%)")
```

```
Starting Deep Dive Grid Search (Train vs Test)…
-----------------------------------------------------------------
Depth  | Rate   | Train Acc  | Test Acc   | Gap (Overfit)
-----------------------------------------------------------------
10     | 0.01   | 90.50%     | 90.36%     | 0.14%
10     | 0.05   | 92.38%     | 92.08%     | 0.29%
10     | 0.1    | 93.23%     | 92.76%     | 0.47%
10     | 0.15   | 93.63%     | 93.04%     | 0.59%
10     | 0.2    | 93.98%     | 93.23%     | 0.75%
15     | 0.01   | 92.43%     | 91.99%     | 0.44%
15     | 0.05   | 93.76%     | 92.96%     | 0.80%
15     | 0.1    | 94.37%     | 93.32%     | 1.05%
```

```
15      | 0.15    | 94.86%      | 93.54%      | 1.31%
15      | 0.2     | 95.13%      | 93.63%      | 1.50%
20      | 0.01    | 93.73%      | 92.83%      | 0.89%
20      | 0.05    | 95.00%      | 93.52%      | 1.48%
20      | 0.1     | 95.35%      | 93.63%      | 1.72%
20      | 0.15    | 95.58%      | 93.70%      | 1.88%
20      | 0.2     | 95.76%      | 93.72%      | 2.04%
25      | 0.01    | 94.36%      | 93.21%      | 1.16%
25      | 0.05    | 95.57%      | 93.62%      | 1.94%
25      | 0.1     | 95.79%      | 93.68%      | 2.11%
25      | 0.15    | 95.95%      | 93.71%      | 2.24%
25      | 0.2     | 96.08%      | 93.71%      | 2.38%
30      | 0.01    | 94.49%      | 93.27%      | 1.22%
30      | 0.05    | 95.76%      | 93.64%      | 2.13%
30      | 0.1     | 96.01%      | 93.68%      | 2.32%
30      | 0.15    | 96.12%      | 93.69%      | 2.43%
30      | 0.2     | 96.24%      | 93.69%      | 2.55%
35      | 0.01    | 94.50%      | 93.27%      | 1.23%
35      | 0.05    | 95.80%      | 93.63%      | 2.17%
35      | 0.1     | 96.09%      | 93.67%      | 2.42%
35      | 0.15    | 96.23%      | 93.69%      | 2.54%
35      | 0.2     | 96.33%      | 93.68%      | 2.65%
-----------------------------------------------------------------
```

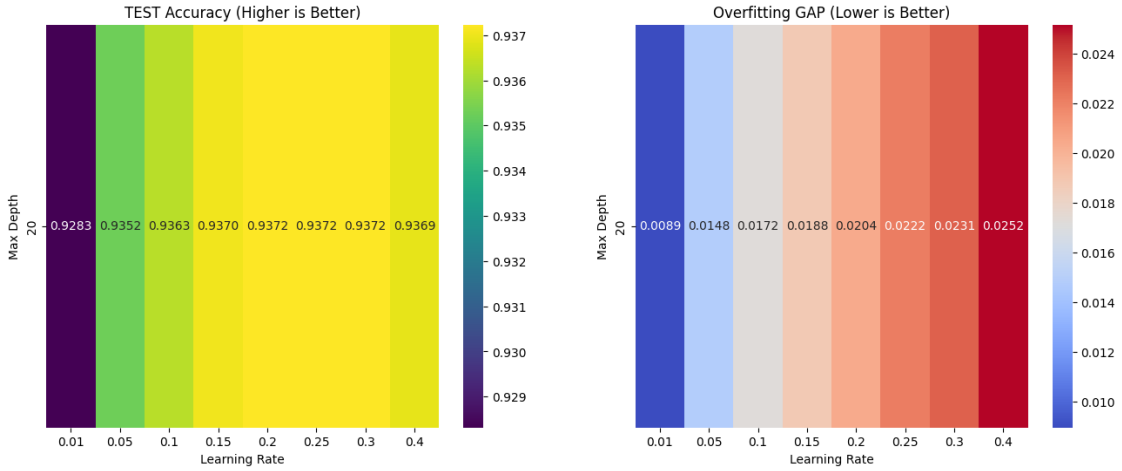



```
  WINNER PARAMETERS:
Best Depth: 20
Best Learning Rate: 0.2
Test Accuracy: 93.72%
Train Accuracy: 95.76% (Gap: 2.04%)
```

[15]:
```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
import xgboost as xgb
from sklearn.model_selection import cross_validate # <--- The new tool

# 1. Search Ranges
param_grid = {
    'max_depth': [20],
    'learning_rate': [0.01, 0.05, 0.1, 0.15, 0.2, 0.25, 0.3, 0.4]
}

results = []

print("Starting Deep Dive Grid Search (Train vs Test)...")
print("-----------------------------------------------------------------")
print(f"{'Depth':<6} | {'Rate':<6} | {'Train Acc':<10} | {'Test Acc':<10} |␣
 ↪{'Gap (Overfit)':<12}")
print("-----------------------------------------------------------------")

# 2. Double Loop
for depth in param_grid['max_depth']:
    for rate in param_grid['learning_rate']:

        model = xgb.XGBClassifier(
            n_estimators=100,
            max_depth=depth,
            learning_rate=rate,
            eval_metric='logloss'
        )

        # Use cross_validate to get BOTH scores
        cv_results = cross_validate(
            model,
            X_train,
            y_train,
            cv=3,
            scoring='accuracy',
            return_train_score=True # <--- This is the magic key
        )

        # Calculate averages
        mean_train = cv_results['train_score'].mean()
        mean_test = cv_results['test_score'].mean()
        gap = mean_train - mean_test

        results.append([depth, rate, mean_train, mean_test, gap])
```

```
        # Print row-by-row
        print(f"{depth:<6} | {rate:<6} | {mean_train*100:.2f}%     |␣
 ↪{mean_test*100:.2f}%     | {gap*100:.2f}%")

print("----------------------------------------------------------------")

# 3. Visualize Results
df_results = pd.DataFrame(results, columns=['Max Depth', 'Learning Rate',␣
 ↪'Train Accuracy', 'Test Accuracy', 'Gap'])

# Create two heatmaps side-by-side
fig, ax = plt.subplots(1, 2, figsize=(16, 6))

# Heatmap 1: Test Accuracy (What we want to maximize)
pivot_test = df_results.pivot(index='Max Depth', columns='Learning Rate',␣
 ↪values='Test Accuracy')
sns.heatmap(pivot_test, annot=True, fmt=".4f", cmap='viridis', ax=ax[0])
ax[0].set_title('TEST Accuracy (Higher is Better)')

# Heatmap 2: Overfitting Gap (What we want to minimize)
pivot_gap = df_results.pivot(index='Max Depth', columns='Learning Rate',␣
 ↪values='Gap')
sns.heatmap(pivot_gap, annot=True, fmt=".4f", cmap='coolwarm', ax=ax[1])
ax[1].set_title('Overfitting GAP (Lower is Better)')

plt.show()
```

```
Starting Deep Dive Grid Search (Train vs Test)…
----------------------------------------------------------------
Depth  | Rate   | Train Acc  | Test Acc   | Gap (Overfit)
----------------------------------------------------------------
20     | 0.01   | 93.73%     | 92.83%     | 0.89%
20     | 0.05   | 95.00%     | 93.52%     | 1.48%
20     | 0.1    | 95.35%     | 93.63%     | 1.72%
20     | 0.15   | 95.58%     | 93.70%     | 1.88%
20     | 0.2    | 95.76%     | 93.72%     | 2.04%
20     | 0.25   | 95.94%     | 93.72%     | 2.22%
20     | 0.3    | 96.03%     | 93.72%     | 2.31%
20     | 0.4    | 96.21%     | 93.69%     | 2.52%
----------------------------------------------------------------
```



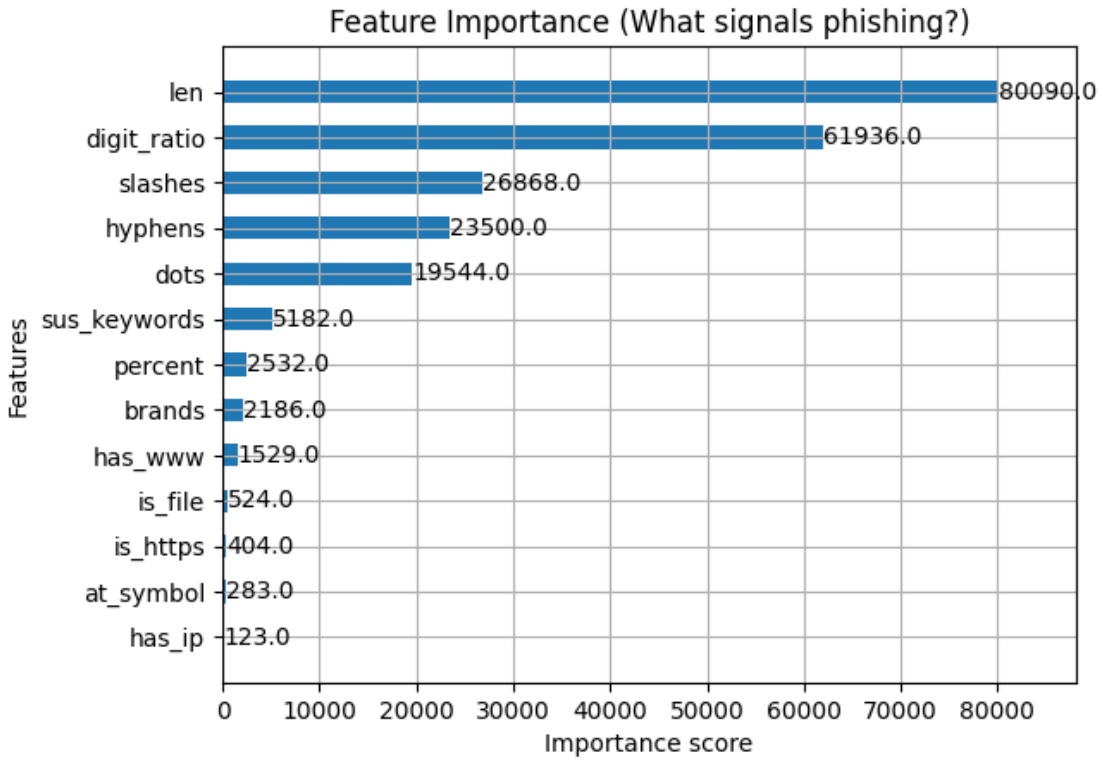

```
 WINNER PARAMETERS:
Best Depth: 20
Best Learning Rate: 0.3
Test Accuracy: 93.72%
Train Accuracy: 96.03% (Gap: 2.31%)
```

[16]:

```
# 1. Retrieve the Winning Parameters from the previous step
# We access the 'best_row' variable we created in the Grid Search cell
final_depth = int(best_row['Max Depth'])
final_rate = best_row['Learning Rate']

print(f" Initializing Final Model with Winner Params: Depth={final_depth},␣
 ↪Rate={final_rate}")

# 2. Train the Final Model on ALL training data
final_model = xgb.XGBClassifier(
    n_estimators=200,          # We assume 200 is good for final polish
    max_depth=final_depth,     # The Winner
    learning_rate=final_rate,  # The Winner
    eval_metric='logloss'
)

final_model.fit(X_train, y_train)

# 3. Final Evaluation on Test Set
# This is the "Moment of Truth" - checking on data the model has NEVER seen
y_pred_final = final_model.predict(X_test)
final_acc = accuracy_score(y_test, y_pred_final)

print(f"\n========================================")
```

```
print(f" FINAL MODEL ACCURACY: {final_acc * 100:.2f}%")
print(f"========================================")

# 4. Feature Importance
# See which new features (like 'dots' or 'keywords') actually mattered
plt.figure(figsize=(10, 8))
xgb.plot_importance(final_model, max_num_features=15, height=0.5,␣
 ↪importance_type='weight', title='Feature Importance (What signals phishing?
 ↪)')
plt.show()

# 5. Save the Brain
final_model.save_model("enhanced_url_classifier.json")
print("Model saved successfully.")
```

```
 Initializing Final Model with Winner Params: Depth=20, Rate=0.3

========================================
 FINAL MODEL ACCURACY: 93.91%
========================================

<Figure size 1000x800 with 0 Axes>
```




```
Model saved successfully.
```

Take 2

[ ]:
```
import math
from collections import Counter
from urllib.parse import urlparse

#              "   "
SUSPICIOUS_TLDS = ['.xyz', '.top', '.club', '.win', '.gq', '.cn', '.tk', '.ml',␣
 ↪'.ga', '.cf', '.buzz']

#                    )     (
SHORTENERS = ['bit.ly', 'goo.gl', 'tinyurl.com', 'is.gd', 'cli.gs', 't.co']

def calculate_entropy(text):
    """
             '  '    .
              ) (            .                .
    """
    if not text:
        return 0
    entropy = 0
    total_len = len(text)
    #
    counts = Counter(text)

    for count in counts.values():
        p = count / total_len
        entropy -= p * math.log2(p)
    return entropy

def advanced_feature_extraction(url):
    url = str(url).lower() #
    features = []

    # ---   '              ) (    ---
    features.append(len(url))
    features.append(url.count('.'))
    features.append(url.count('-'))
    features.append(url.count('@'))
    features.append(url.count('%'))
    features.append(url.count('/'))

    # ---   '             ---

    # 1.          )      , (
    digits = sum(c.isdigit() for c in url)
    letters = sum(c.isalpha() for c in url)
```

```
    features.append(digits / (letters + 1))

    # 2. NEW:        )     (
    #                      ,
    try:
        domain = urlparse(url).netloc
        if not domain: domain = url
    except:
        domain = url
    features.append(calculate_entropy(domain))

    # ---   '        ---

    # 3. NEW:            ? (TLD)
    is_sus_tld = 1 if any(url.endswith(tld) for tld in SUSPICIOUS_TLDS) else 0
    features.append(is_sus_tld)

    # 4. NEW:                ?
    is_shortener = 1 if any(s in url for s in SHORTENERS) else 0
    features.append(is_shortener)

    #        (   )
    suspicious_words = ['login', 'signin', 'bank', 'secure', 'account',␣
↪'update',
                        'verify', 'confirm', 'wallet', 'crypto', 'bonus',␣
↪'free', 'ebay']
    features.append(sum(1 for word in suspicious_words if word in url))

    #     (   )
    brands = ['paypal', 'apple', 'google', 'microsoft', 'facebook', 'netflix',␣
↪'amazon']
    features.append(sum(1 for brand in brands if brand in url))

    #         (   )
    dangerous_ext = ['.exe', '.zip', '.tar', '.js', '.vbs', '.apk']
    features.append(1 if any(ext in url for ext in dangerous_ext) else 0)

    # HTTPS WWW-    ),          ,     (
    features.append(1 if 'https' in url else 0)
    features.append(1 if 'www.' in url else 0)

    return features

print("New Feature Extraction Logic Loaded!  ")
```

[19]:
```
print("  Applying NEW feature extraction logic to the dataset...")
```

```
print("   (Calculating Entropy & checking TLDs for all URLs - this takes a␣
 ↪moment)")

# 1.                    URL-
X_new_features = df['url'].apply(lambda x: pd.
 ↪Series(advanced_feature_extraction(x)))

# 2.               )           (
feature_names = [
    'len', 'dots', 'hyphens', 'at_symbol', 'percent', 'slashes',
    'digit_ratio',       #
    'entropy',           # <---   !
    'is_sus_tld',        # <---   !
    'is_shortener',      # <---   !
    'sus_keywords', 'brands', 'dangerous_ext', 'is_https', 'has_www' #
]

X = pd.DataFrame(X_new_features.values, columns=feature_names)
y = df['label'] # Y-

# 3.        Train-  Test-   )    '           (
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=42)

print(" Data updated! Now the model can see the Entropy and TLDs.")
print(f"   New Feature Matrix Shape: {X.shape}")
```

```
 Applying NEW feature extraction logic to the dataset…
   (Calculating Entropy & checking TLDs for all URLs - this takes a moment)
 Data updated! Now the model can see the Entropy and TLDs.
   New Feature Matrix Shape: (651191, 15)
```

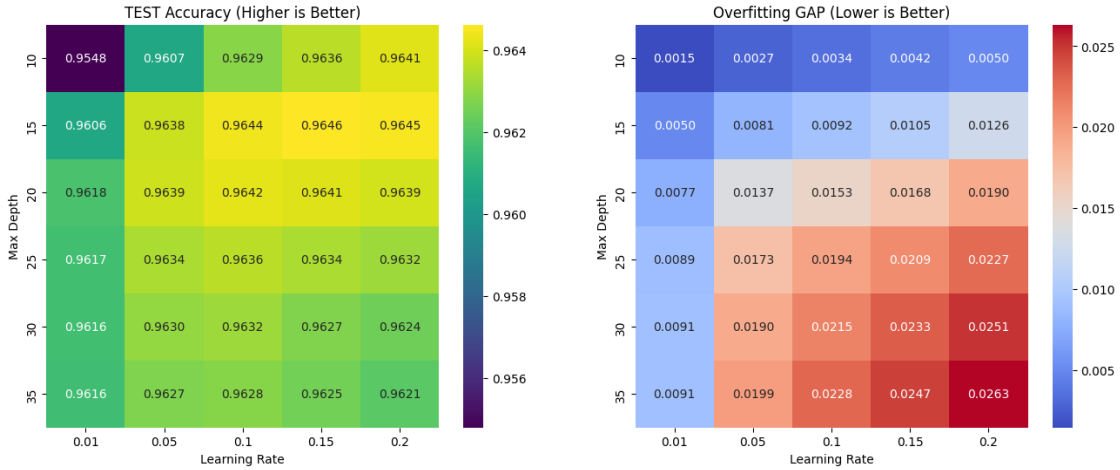
[20]:
```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
import xgboost as xgb
from sklearn.model_selection import cross_validate # <--- The new tool

# 1. Search Ranges
param_grid = {
    'max_depth': [10, 15, 20, 25, 30, 35],
    'learning_rate': [0.01, 0.05, 0.1, 0.15, 0.2]
}

results = []

print("Starting Deep Dive Grid Search (Train vs Test)...")
```

```
print("-----------------------------------------------------------------")
print(f"{'Depth':<6} | {'Rate':<6} | {'Train Acc':<10} | {'Test Acc':<10} |␣
 ↪{'Gap (Overfit)':<12}")
print("-----------------------------------------------------------------")

# 2. Double Loop
for depth in param_grid['max_depth']:
    for rate in param_grid['learning_rate']:

        model = xgb.XGBClassifier(
            n_estimators=100,
            max_depth=depth,
            learning_rate=rate,
            eval_metric='logloss'
        )

        # Use cross_validate to get BOTH scores
        cv_results = cross_validate(
            model,
            X_train,
            y_train,
            cv=3,
            scoring='accuracy',
            return_train_score=True # <--- This is the magic key
        )

        # Calculate averages
        mean_train = cv_results['train_score'].mean()
        mean_test = cv_results['test_score'].mean()
        gap = mean_train - mean_test

        results.append([depth, rate, mean_train, mean_test, gap])

        # Print row-by-row
        print(f"{depth:<6} | {rate:<6} | {mean_train*100:.2f}%     |␣
 ↪{mean_test*100:.2f}%     | {gap*100:.2f}%")

print("-----------------------------------------------------------------")

# 3. Visualize Results
df_results = pd.DataFrame(results, columns=['Max Depth', 'Learning Rate',␣
 ↪'Train Accuracy', 'Test Accuracy', 'Gap'])

# Create two heatmaps side-by-side
fig, ax = plt.subplots(1, 2, figsize=(16, 6))

# Heatmap 1: Test Accuracy (What we want to maximize)
```

```
pivot_test = df_results.pivot(index='Max Depth', columns='Learning Rate',␣
 ↪values='Test Accuracy')
sns.heatmap(pivot_test, annot=True, fmt=".4f", cmap='viridis', ax=ax[0])
ax[0].set_title('TEST Accuracy (Higher is Better)')

# Heatmap 2: Overfitting Gap (What we want to minimize)
pivot_gap = df_results.pivot(index='Max Depth', columns='Learning Rate',␣
 ↪values='Gap')
sns.heatmap(pivot_gap, annot=True, fmt=".4f", cmap='coolwarm', ax=ax[1])
ax[1].set_title('Overfitting GAP (Lower is Better)')

plt.show()

# 4. Auto-select the winner (Based on best TEST accuracy)
best_row = df_results.loc[df_results['Test Accuracy'].idxmax()]
print("\n WINNER PARAMETERS:")
print(f"Best Depth: {int(best_row['Max Depth'])}")
print(f"Best Learning Rate: {best_row['Learning Rate']}")
print(f"Test Accuracy: {best_row['Test Accuracy']*100:.2f}%")
print(f"Train Accuracy: {best_row['Train Accuracy']*100:.2f}% (Gap:␣
 ↪{best_row['Gap']*100:.2f}%)")
```

```
Starting Deep Dive Grid Search (Train vs Test)…
----------------------------------------------------------------------
Depth  | Rate   | Train Acc  | Test Acc   | Gap (Overfit)
----------------------------------------------------------------------
10     | 0.01   | 95.62%     | 95.48%     | 0.15%
10     | 0.05   | 96.34%     | 96.07%     | 0.27%
10     | 0.1    | 96.63%     | 96.29%     | 0.34%
10     | 0.15   | 96.78%     | 96.36%     | 0.42%
10     | 0.2    | 96.91%     | 96.41%     | 0.50%
15     | 0.01   | 96.56%     | 96.06%     | 0.50%
15     | 0.05   | 97.19%     | 96.38%     | 0.81%
15     | 0.1    | 97.36%     | 96.44%     | 0.92%
15     | 0.15   | 97.52%     | 96.46%     | 1.05%
15     | 0.2    | 97.71%     | 96.45%     | 1.26%
20     | 0.01   | 96.95%     | 96.18%     | 0.77%
20     | 0.05   | 97.76%     | 96.39%     | 1.37%
20     | 0.1    | 97.95%     | 96.42%     | 1.53%
20     | 0.15   | 98.09%     | 96.41%     | 1.68%
20     | 0.2    | 98.29%     | 96.39%     | 1.90%
25     | 0.01   | 97.06%     | 96.17%     | 0.89%
25     | 0.05   | 98.07%     | 96.34%     | 1.73%
25     | 0.1    | 98.30%     | 96.36%     | 1.94%
25     | 0.15   | 98.43%     | 96.34%     | 2.09%
25     | 0.2    | 98.58%     | 96.32%     | 2.27%
30     | 0.01   | 97.07%     | 96.16%     | 0.91%
```

```
30      | 0.05    | 98.20%      | 96.30%      | 1.90%
30      | 0.1     | 98.46%      | 96.32%      | 2.15%
30      | 0.15    | 98.61%      | 96.27%      | 2.33%
30      | 0.2     | 98.75%      | 96.24%      | 2.51%
35      | 0.01    | 97.07%      | 96.16%      | 0.91%
35      | 0.05    | 98.26%      | 96.27%      | 1.99%
35      | 0.1     | 98.57%      | 96.28%      | 2.28%
35      | 0.15    | 98.72%      | 96.25%      | 2.47%
35      | 0.2     | 98.84%      | 96.21%      | 2.63%
-------------------------------------------------------------
```



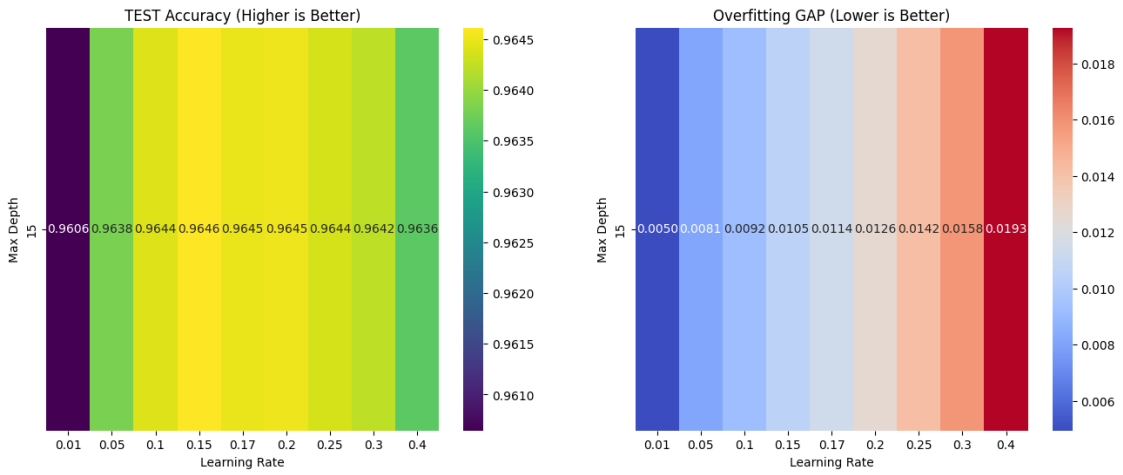

```
 WINNER PARAMETERS:
Best Depth: 15
Best Learning Rate: 0.15
Test Accuracy: 96.46%
Train Accuracy: 97.52% (Gap: 1.05%)
```

[21]:
```python
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
import xgboost as xgb
from sklearn.model_selection import cross_validate # <--- The new tool

# 1. Search Ranges
param_grid = {
    'max_depth': [15],
    'learning_rate': [0.01, 0.05, 0.1, 0.15, 0.17, 0.2, 0.25, 0.3, 0.4]
}

results = []
```

```
print("Starting Deep Dive Grid Search (Train vs Test)...")
print("-----------------------------------------------------------------")
print(f"{'Depth':<6} | {'Rate':<6} | {'Train Acc':<10} | {'Test Acc':<10} |␣
 ↪{'Gap (Overfit)':<12}")
print("-----------------------------------------------------------------")

# 2. Double Loop
for depth in param_grid['max_depth']:
    for rate in param_grid['learning_rate']:

        model = xgb.XGBClassifier(
            n_estimators=100,
            max_depth=depth,
            learning_rate=rate,
            eval_metric='logloss'
        )

        # Use cross_validate to get BOTH scores
        cv_results = cross_validate(
            model,
            X_train,
            y_train,
            cv=3,
            scoring='accuracy',
            return_train_score=True # <--- This is the magic key
        )

        # Calculate averages
        mean_train = cv_results['train_score'].mean()
        mean_test = cv_results['test_score'].mean()
        gap = mean_train - mean_test

        results.append([depth, rate, mean_train, mean_test, gap])

        # Print row-by-row
        print(f"{depth:<6} | {rate:<6} | {mean_train*100:.2f}%     |␣
 ↪{mean_test*100:.2f}%     | {gap*100:.2f}%")

print("-----------------------------------------------------------------")

# 3. Visualize Results
df_results = pd.DataFrame(results, columns=['Max Depth', 'Learning Rate',␣
 ↪'Train Accuracy', 'Test Accuracy', 'Gap'])

# Create two heatmaps side-by-side
fig, ax = plt.subplots(1, 2, figsize=(16, 6))
```

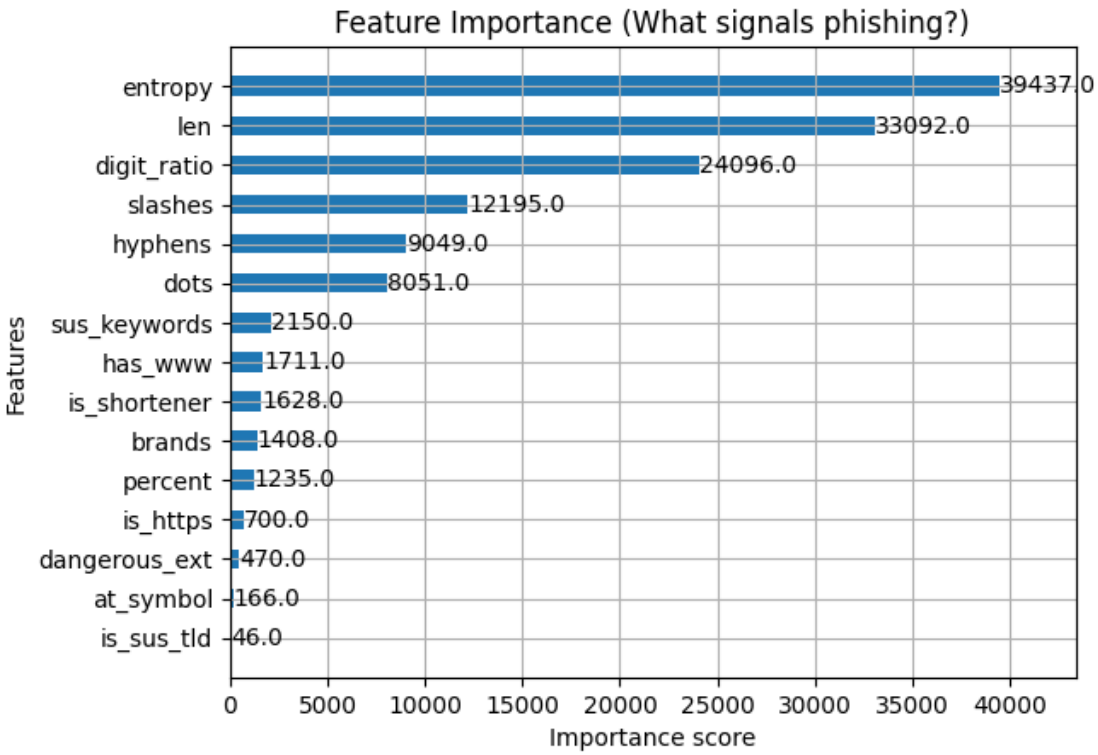
```python
# Heatmap 1: Test Accuracy (What we want to maximize)
pivot_test = df_results.pivot(index='Max Depth', columns='Learning Rate',␣
 ↪values='Test Accuracy')
sns.heatmap(pivot_test, annot=True, fmt=".4f", cmap='viridis', ax=ax[0])
ax[0].set_title('TEST Accuracy (Higher is Better)')

# Heatmap 2: Overfitting Gap (What we want to minimize)
pivot_gap = df_results.pivot(index='Max Depth', columns='Learning Rate',␣
 ↪values='Gap')
sns.heatmap(pivot_gap, annot=True, fmt=".4f", cmap='coolwarm', ax=ax[1])
ax[1].set_title('Overfitting GAP (Lower is Better)')

plt.show()
```

```
Starting Deep Dive Grid Search (Train vs Test)…
----------------------------------------------------------------------
Depth  | Rate   | Train Acc  | Test Acc   | Gap (Overfit)
----------------------------------------------------------------------
15     | 0.01   | 96.56%     | 96.06%     | 0.50%
15     | 0.05   | 97.19%     | 96.38%     | 0.81%
15     | 0.1    | 97.36%     | 96.44%     | 0.92%
15     | 0.15   | 97.52%     | 96.46%     | 1.05%
15     | 0.17   | 97.59%     | 96.45%     | 1.14%
15     | 0.2    | 97.71%     | 96.45%     | 1.26%
15     | 0.25   | 97.86%     | 96.44%     | 1.42%
15     | 0.3    | 98.01%     | 96.42%     | 1.58%
15     | 0.4    | 98.29%     | 96.36%     | 1.93%
----------------------------------------------------------------------
```



[22]:
```
# 1. Retrieve the Winning Parameters from the previous step
# We access the 'best_row' variable we created in the Grid Search cell
final_depth = 15
final_rate = 0.15

print(f" Initializing Final Model with Winner Params: Depth={final_depth},␣
 ↪Rate={final_rate}")

# 2. Train the Final Model on ALL training data
final_model = xgb.XGBClassifier(
    n_estimators=200,          # We assume 200 is good for final polish
    max_depth=final_depth,     # The Winner
    learning_rate=final_rate,  # The Winner
    eval_metric='logloss'
)

final_model.fit(X_train, y_train)

# 3. Final Evaluation on Test Set
# This is the "Moment of Truth" - checking on data the model has NEVER seen
y_pred_final = final_model.predict(X_test)
final_acc = accuracy_score(y_test, y_pred_final)

print(f"\n========================================")
print(f" FINAL MODEL ACCURACY: {final_acc * 100:.2f}%")
print(f"========================================")

# 4. Feature Importance
# See which new features (like 'dots' or 'keywords') actually mattered
plt.figure(figsize=(10, 8))
xgb.plot_importance(final_model, max_num_features=15, height=0.5,␣
 ↪importance_type='weight', title='Feature Importance (What signals phishing?
 ↪)')
plt.show()

# 5. Save the Brain
final_model.save_model("enhanced_url_classifier.json")
print("Model saved successfully.")
```

```
 Initializing Final Model with Winner Params: Depth=15, Rate=0.15

========================================
 FINAL MODEL ACCURACY: 96.59%
========================================

<Figure size 1000x800 with 0 Axes>
```




```
Model saved successfully.
```

[ ]: